

ĐỒ ÁN CUỐI KỲ

Crypto Strategy Lab – Nền tảng phân tích, kết hợp và đánh giá chiến lược giao dịch Crypto

1. Bối cảnh bài toán

Thị trường cryptocurrency như Bitcoin, Ethereum... hoạt động liên tục 24/7. Giá của các tài sản thay đổi theo thời gian và thường được biểu diễn bằng **biểu đồ nến – Candlestick Chart**.

Ví dụ với cặp giao dịch **BTC/USDT**, một cây nến 5 phút chứa:

- Open: giá BTC ở đầu 5 phút.
- High: giá cao nhất trong 5 phút.
- Low: giá thấp nhất trong 5 phút.
- Close: giá cuối 5 phút.
- Volume: khối lượng giao dịch trong 5 phút.

Ví dụ:

BTCUSDT – khung 5 phút

09:00

Open = 118,000

High = 118,200

Low = 117,900

Close = 118,150

Volume = 125 BTC

Các trader thường sử dụng nhiều phương pháp phân tích kỹ thuật như:

- Moving Average – MA
- RSI
- Bollinger Bands
- Support/Resistance
- Smart Money Concepts – SMC
- Wyckoff
- ...

để tìm thời điểm thích hợp để **Buy, Sell hoặc không giao dịch**.

Tuy nhiên, một strategy đơn lẻ thường không hoạt động tốt trong mọi điều kiện thị trường.

Ví dụ:

MA:

Tốt khi thị trường có xu hướng.

Kém khi thị trường đi ngang.

RSI:

Có thể hữu ích khi phát hiện quá mua/quá bán.

Nhưng có thể tạo nhiều tín hiệu sai khi thị trường đang có trend mạnh.

Support/Resistance:

Có thể tìm vùng giá quan trọng.

Nhưng việc xác định vùng hỗ trợ/kháng cự có thể phụ thuộc vào thuật toán.

Vì vậy, câu hỏi chính của đề án là:

Có thể xây dựng một hệ thống cho phép bổ sung nhiều strategy khác nhau, tự động kết hợp chúng thành các strategy phức hợp, đánh giá hiệu quả và liên tục tìm ra những tổ hợp strategy tốt nhất hay không?

2. Mục tiêu tổng thể

Xây dựng một nền tảng **Crypto Strategy Lab** có khả năng:

1. Nhận dữ liệu thị trường cryptocurrency từ Binance.
2. Hiển thị biểu đồ giá realtime.
3. Theo dõi đồng thời tối đa 4 khung thời gian.
4. Cho phép bổ sung các strategy phân tích kỹ thuật.
5. Cho phép kết hợp nhiều strategy thành một chiến lược tổng hợp.
6. Backtest các chiến lược trên dữ liệu lịch sử.
7. Xếp hạng các strategy dựa trên hiệu quả giao dịch.
8. Tự động tìm kiếm các combination strategy tốt hơn.
9. Visualize tín hiệu và giao dịch lên biểu đồ.
10. Thu thập tin tức liên quan đến coin/pair.
11. Phân tích sentiment của tin tức bằng mô hình Machine Learning.
12. Thiết kế hệ thống sao cho có thể mở rộng trong tương lai mà không phải sửa đổi toàn bộ hệ thống.

Trọng tâm của đề án là Kiến trúc phần mềm, không phải tìm ra strategy đầu tư tốt nhất.

3. Một ví dụ tổng thể

Giả sử người dùng chọn:

Pair: BTCUSDT

Timeframes:

5m

15m

1h

4h

Dashboard hiển thị 4 biểu đồ:

```
+-----+-----+
| BTCUSDT - 5m   | BTCUSDT - 15m   |
|               |               |
| Candlestick    | Candlestick    |
+-----+-----+

+-----+-----+
| BTCUSDT - 1h   | BTCUSDT - 4h    |
|               |               |
| Candlestick    | Candlestick    |
+-----+-----+
```

Người dùng có thể đổi:

5m → 1m
15m → 30m
1h → 2h
4h → 1d

mà không phải reload toàn bộ hệ thống.

Sau đó người dùng bật:

MA
RSI
Bollinger Bands
Support/Resistance

Hệ thống có thể tạo:

Strategy A
MA + RSI

Strategy B
MA + Bollinger

Strategy C
RSI + Support/Resistance

Strategy D
MA + RSI + Support/Resistance

Strategy E
MA + RSI + Bollinger + Support/Resistance

Sau khi backtest, hệ thống có thể đưa ra:

Rank	Strategy	Profit	Win Rate	Max Drawdown
1	MA + RSI + SR	+18.2%	61%	-6.1%
2	MA + Bollinger	+15.7%	58%	-8.4%
3	RSI + SR	+13.1%	64%	-7.2%

Đây chính là **Leaderboard**.

4. Module 1 – Realtime Market Data

Hệ thống cần lấy dữ liệu giá crypto từ Binance.

Có hai loại dữ liệu chính.

Historical Data

Dữ liệu trong quá khứ.

Ví dụ:

BTCUSDT
01/07 → 30/07

1 phút
5 phút
15 phút
1 giờ

4 giờ
1 ngày

Dữ liệu này phù hợp cho:

- backtesting;
- tính indicator;
- huấn luyện ML;
- phân tích lịch sử.

Realtime Data

Dữ liệu giá đang thay đổi tại thời điểm hiện tại.

Ví dụ:

```
09:10:01 BTC = 118,021
09:10:02 BTC = 118,028
09:10:03 BTC = 118,017
...
```

Frontend cần nhận cập nhật mà không liên tục gọi:

```
GET /price
GET /price
GET /price
```

Sinh viên nên nghiên cứu kiến trúc phù hợp như:

```

Binance
  ↓
Market Data Adapter
  ↓
Event / Stream
  ↓
Backend
  ↓
WebSocket
  ↓
Frontend
```

Yêu cầu kiến trúc

Không được để frontend phụ thuộc trực tiếp vào cấu trúc dữ liệu Binance.

Ví dụ không nên:

```
Frontend → Binance API
```

Nên:

```
Frontend
  ↓
Market Data Service
  ↓
Binance Adapter
  ↓
Binance
```

Nhờ đó sau này có thể bổ sung:

```
BinanceAdapter
OKXAdapter
BybitAdapter
CoinbaseAdapter
```

mà frontend không phải thay đổi.

5. Module 2 – Multi-Timeframe Chart

Hệ thống phải hỗ trợ tối đa **4 chart trên một màn hình**.

Ví dụ:

```
BTCUSDT
```

```
Chart 1 → 5m
Chart 2 → 15m
Chart 3 → 1h
Chart 4 → 4h
```

Mỗi chart phải có thể thay đổi timeframe riêng.

Ví dụ:

Chart 1

Pair:
BTCUSDT

Timeframe:
[1m] [5m] [15m] [1h] [4h] [1d]

Nếu người dùng đổi:

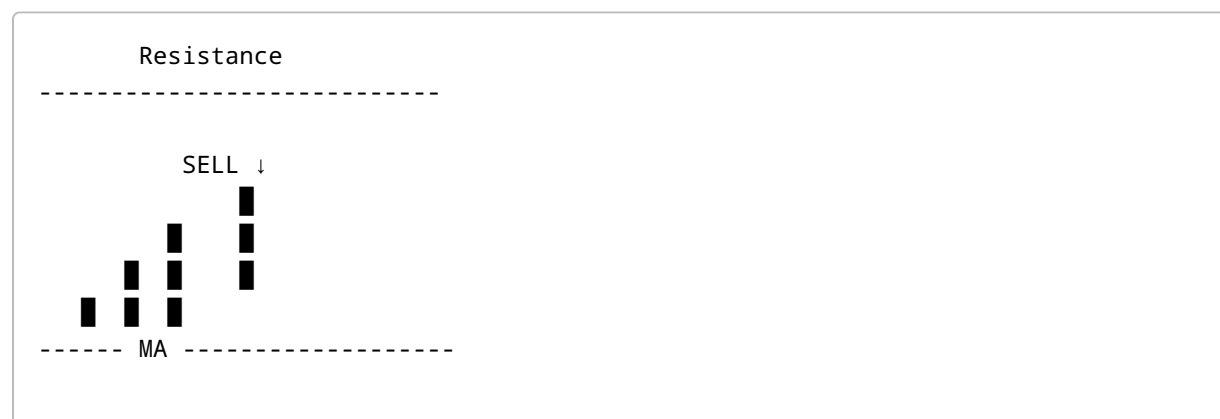
5m → 1h

thì chỉ Chart 1 cần đổi dữ liệu.

Có thể visualize

- Candlestick.
- Volume.
- MA.
- Bollinger Bands.
- vùng Support.
- vùng Resistance.
- Buy Signal.
- Sell Signal.
- điểm Entry.
- Stop Loss.
- Take Profit.

Ví dụ:



↑ BUY

Support

6. Module 3 – Strategy Engine

Đây là một module quan trọng nhất của hệ thống.

Một **strategy** nhận dữ liệu thị trường và tạo ra một tín hiệu.

Có thể chuẩn hóa tín hiệu thành:

BUY
SELL
HOLD

hoặc:

LONG
SHORT
NONE

Ví dụ:

```
interface Strategy {  
  
    analyze(context)  
  
    return:  
        BUY  
        SELL  
        HOLD  
}
```

`context` có thể chứa:

price
volume
candles


```
timeframe
indicators
market state
sentiment
...
```

7. Strategy ví dụ 1 – Moving Average

Moving Average – MA là giá trung bình của một khoảng thời gian.

Ví dụ:

```
MA20 = trung bình giá của 20 candles gần nhất.
MA50 = trung bình giá của 50 candles gần nhất.
```

Strategy đơn giản:

```
Nếu MA20 cắt lên MA50
→ BUY

Nếu MA20 cắt xuống MA50
→ SELL
```

Có thể implement:

```
MAStrategy
    fastPeriod = 20
    slowPeriod = 50
```

Điều quan trọng về kiến trúc:

`MAStrategy` chỉ nên chịu trách nhiệm về logic MA.

Không nên chứa:

```
code gọi Binance
code lưu database
code vẽ chart
code gửi notification
```

8. Strategy ví dụ 2 – RSI

RSI có giá trị từ:

0 → 100

Ví dụ một rule đơn giản:

```
RSI < 30
→ Oversold
→ BUY

RSI > 70
→ Overbought
→ SELL
```

Có thể xây dựng:

```
RSIStrategy
    period = 14
    buyThreshold = 30
    sellThreshold = 70
```

Như vậy có thể thử:

```
RSI(14, 30, 70)
RSI(14, 25, 75)
RSI(21, 30, 70)
```

9. Strategy ví dụ 3 – Bollinger Bands

Bollinger Bands tạo ba đường:

```
Upper Band
Middle Band
Lower Band
```

Ví dụ strategy:

Price < Lower Band
→ BUY

Price > Upper Band
→ SELL

Hoặc strategy khác:

Price breakout Upper Band
→ BUY

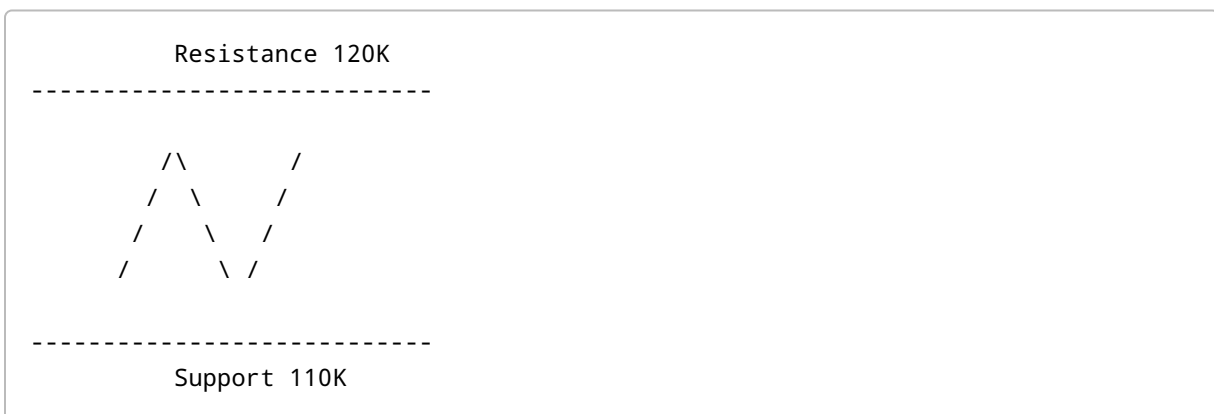
Như vậy cùng một indicator có thể sinh ra nhiều strategy khác nhau.

10. Strategy ví dụ 4 – Support/Resistance

Support là vùng giá mà giá trước đây thường ngừng giảm.

Resistance là vùng mà giá trước đây thường gặp khó khăn khi tăng tiếp.

Ví dụ:



Một strategy có thể là:

Price gần Support
→ BUY

Price gần Resistance
→ SELL

Hoặc:

Price breakout Resistance
→ BUY

11. Strategy nâng cao – SMC, Wyckoff

Sinh viên không bắt buộc phải xây dựng đầy đủ các phương pháp phức tạp này.

Mục tiêu là chứng minh kiến trúc có khả năng hỗ trợ chúng.

Ví dụ:

```
Strategy
  ↑
-----
MA Strategy
RSI Strategy
Bollinger Strategy
SMC Strategy
Wyckoff Strategy
Sentiment Strategy
```

Thêm một strategy mới không được yêu cầu sửa toàn bộ Strategy Engine.

Đây chính là yêu cầu về:

Extensibility – khả năng mở rộng hệ thống.

12. Module 4 – Strategy Plugin

Một yêu cầu quan trọng:

Hệ thống phải cho phép bổ sung strategy mới dễ dàng.

Ví dụ ban đầu hệ thống có:

```
strategies/
```

```
MA  
RSI  
Bollinger
```

Nhóm phát triển thêm:

```
SupportResistance
```

Lý tưởng nhất, Strategy Engine chỉ cần đăng ký:

```
register(SupportResistance)
```

thay vì phải sửa:

```
if strategy == MA ...  
else if strategy == RSI ...  
else if strategy == Bollinger ...  
else if strategy == SR ...
```

Sinh viên cần nghiên cứu các architectural/design pattern thích hợp như:

- Strategy Pattern.
- Plugin Architecture.
- Factory.
- Registry.
- Dependency Injection.

Không bắt buộc phải sử dụng đúng một pattern cụ thể.

Quan trọng là phải giải thích được:

Vì sao kiến trúc của nhóm có thể thêm strategy mới mà ảnh hưởng tối thiểu đến code hiện tại?

13. Module 5 – Composite Strategy

Đây là phần trung tâm của bài toán.

Giả sử có:

```
MA
RSI
Bollinger
SupportResistance
```

Ta có thể tạo:

```
MA + RSI
MA + Bollinger
MA + SR
RSI + Bollinger
RSI + SR
MA + RSI + SR
...
```

Nhưng câu hỏi quan trọng là:

Khi các strategy đưa ra tín hiệu khác nhau thì kết hợp thế nào?

Ví dụ:

```
MA → BUY
RSI → BUY
SR → HOLD
```

Có thể dùng Majority Vote:

```
BUY = 2
HOLD = 1

→ BUY
```

Một trường hợp khác:

```
MA → BUY
RSI → SELL
SR → BUY
```

→ BUY

14. Weighted Combination

Không nhất thiết strategy nào cũng có trọng số giống nhau.

Ví dụ:

```
MA    = 0.2
RSI    = 0.3
SR    = 0.5
```

Nếu encode:

```
BUY    = +1
HOLD    = 0
SELL    = -1
```

và:

```
MA    → BUY
RSI    → SELL
SR    → BUY
```

ta có:

```
Score

= MA × 0.2
+ RSI × 0.3
+ SR × 0.5

= 1×0.2
+ (-1)×0.3
+ 1×0.5

= 0.4
```

Quy định:

```
score > 0.3  
→ BUY  
  
score < -0.3  
→ SELL  
  
còn lại  
→ HOLD
```

Đây chỉ là một ví dụ. Nhóm được quyền thiết kế phương pháp combination riêng.

15. Module 6 – Strategy Search Engine

Nếu có nhiều strategy, số tổ hợp có thể tăng rất nhanh.

Ví dụ chỉ có 4 strategy:

```
MA  
RSI  
BB  
SR
```

đã có thể tạo:

```
MA + RSI  
MA + BB  
MA + SR  
  
RSI + BB  
RSI + SR  
  
BB + SR  
  
MA + RSI + BB  
MA + RSI + SR  
...
```

Nếu mỗi strategy lại có nhiều parameter:

MA :

10/20

20/50

50/200

RSI :

14/30/70

14/20/80

21/30/70

không gian tìm kiếm sẽ lớn hơn rất nhiều.

Hệ thống cần cung cấp một **Strategy Search Engine**.

16. Cách tìm kiếm 1 – Random Search

Cách đơn giản nhất:

Random một tổ hợp.

Ví dụ:

```
Loop 1
MA + RSI

Loop 2
BB + SR

Loop 3
MA + RSI + SR

Loop 4
MA + BB + SR

...
```

Mỗi combination được:

```
generate
↓
backtest
↓
evaluate
```

↓
rank

17. Cách tìm kiếm 2 – Domain-guided Search

Thay vì random hoàn toàn, có thể dựa trên đặc điểm domain.

Ví dụ phân nhóm:

```
Trend:
MA
MACD

Momentum:
RSI
Stochastic

Volatility:
Bollinger
ATR

Structure:
Support/Resistance
SMC
Wyckoff

Information:
News Sentiment
```

Có thể đặt rule:

```
Một composite strategy phải lấy:

1 Trend Strategy
+
1 Momentum Strategy
+
1 Structure Strategy
```

Ví dụ:

MA
+
RSI
+
Support Resistance

thay vì:

MA10
+
MA20
+
MA50

Nhóm cần giải thích:

Domain knowledge đã được đưa vào quá trình search như thế nào?

18. Cách tìm kiếm nâng cao

Nhóm có thể nghiên cứu thêm:

Genetic Algorithm
Bayesian Optimization
Evolutionary Search
Reinforcement Learning
LLM-generated Strategy
Agent-based Search
AlphaEvolve-style optimization
Loop Engineering

Đây là **phần mở rộng**, không bắt buộc.

19. Module 7 – Backtesting Engine

Backtesting nghĩa là giả lập:

Nếu sử dụng strategy này trong quá khứ thì kết quả sẽ như thế nào?

Ví dụ dữ liệu:

```
01/01 BTC = $80,000
...
01/03 BTC = $95,000
```

Strategy tạo:

```
05/01 BUY @82,000
12/01 SELL @86,000

22/01 BUY @88,000
31/01 SELL @87,000
```

Backtesting Engine sẽ giả lập các giao dịch này.

Ví dụ:

```
Trade 1
Buy 82K
Sell 86K
→ Profit
```

```
Trade 2
Buy 88K
Sell 87K
→ Loss
```

20. Không chỉ đánh giá Profit

Strategy không được đánh giá chỉ bằng:

Total Profit

Ví dụ:

Strategy A

Profit = +30%

nhưng từng có lúc:
-45%

Strategy B

Profit = +25%

nhưng Max Drawdown:
-8%

Strategy B có thể ổn định hơn Strategy A.

Hệ thống nên cung cấp một số metrics như:

Total Return
Profit/Loss
Win Rate
Number of Trades
Maximum Drawdown
Profit Factor
Sharpe Ratio

Không yêu cầu sinh viên phải hiểu sâu tài chính định lượng.

Nhưng cần hiểu:

Strategy Evaluation phải tách biệt khỏi Strategy Implementation.

21. Module 8 – Leaderboard

Sau mỗi lần backtest, kết quả được đưa vào Leaderboard.

Ví dụ:

Rank	Strategy	Return	Win Rate	MDD	Trades
1	MA+RSI+SR	24.2%	62%	-7.1%	81
2	MA+BB	21.7%	55%	-8.4%	105
3	RSI+SR	18.4%	64%	-6.7%	52
4	MA	9.1%	48%	-14.2%	140

Có thể cho phép:

Sort by:
Return
Win Rate
Max Drawdown
Sharpe

Hoặc định nghĩa:

Overall Score

Ví dụ:

Score =
 $0.5 \times \text{Return}$
 $+ 0.2 \times \text{WinRate}$
 $+ 0.3 \times \text{RiskScore}$

Nhóm phải trình bày rõ cách tính.

22. Top-K Strategies

Hệ thống không nhất thiết giữ tất cả strategy tốt nhất lên màn hình.

Ví dụ:

Top K = 10

Leaderboard luôn hiển thị:

Top 10 strategies hiện tại

Một candidate mới:

MA20 + RSI14 + SR

được backtest.

Nếu score:

82.1

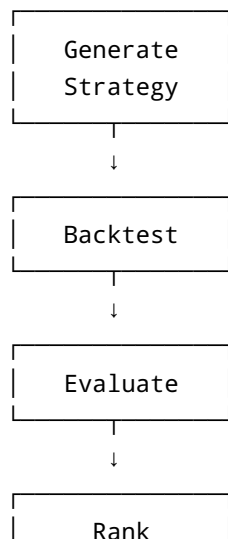
cao hơn strategy đứng thứ 10:

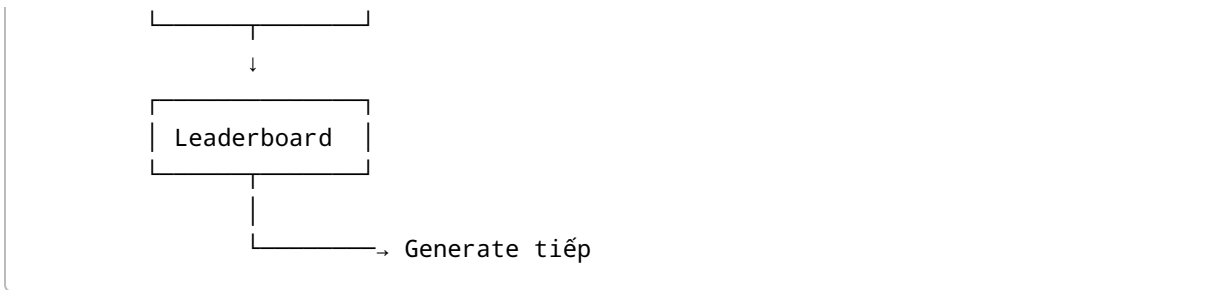
78.4

thì strategy mới được đưa vào Leaderboard.

23. Module 9 – Continuous Strategy Loop

Hệ thống có thể chạy một vòng loop ngàm:





Ví dụ:

```
#182  
MA20 + RSI14  
Score = 71  
  
#183  
MA20 + Bollinger  
Score = 68  
  
#184  
MA50 + RSI21 + SR  
Score = 84  
→ New Top Strategy  
  
#185  
...
```

Loop có thể chạy:

```
100 candidate  
  
hoặc  
  
1 giờ  
  
hoặc  
  
đến khi không cải thiện sau 50 iterations.
```

Nhóm phải thiết kế **Stop Condition**.

Không được để:


```
while(true)
```

chạy vô hạn mà không kiểm soát.

24. Vì sao phần Loop quan trọng đối với Kiến trúc phần mềm?

Một implementation kém có thể viết:

```
for 100000 strategies:

    calculate indicator
    backtest
    save DB
    update UI
```

Tất cả nằm trong một function.

Implementation tốt nên tách:

```
Strategy Generator
  ↓
Strategy Queue
  ↓
Backtest Worker
  ↓
Evaluator
  ↓
Ranking Service
  ↓
Leaderboard
```

Qua đó có thể:

- chạy nhiều worker;
- retry khi worker lỗi;
- pause loop;
- resume loop;
- theo dõi tiến trình;
- thay search algorithm;

- scale trong tương lai.

25. Visualization Strategy

Không chỉ hiển thị:

Profit = +20%

mà phải cho phép người dùng hiểu strategy đã làm gì.

Ví dụ:

BTCUSDT 15m

SELL
↓



↑
BUY

MA -----

Support =====

Người dùng click:

Strategy:
MA20 + RSI14 + SupportResistance

chart hiển thị:

MA20

RSI signals

Support zones

Buy points

Sell points

26. Trade Detail

Người dùng có thể xem bảng:

#	Entry Time	Entry	Exit Time	Exit	Result
1	01/07 08:00	108K	01/07 15:00	110K	+1.85%
2	02/07 10:00	111K	02/07 18:00	110K	-0.90%
3	04/07 07:00	109K	05/07 12:00	114K	+4.58%

Click Trade #3 thì chart có thể highlight:

ENTRY ↑

...

EXIT ↓

27. Module 10 – News Crawler

Giá cryptocurrency không chỉ phụ thuộc vào biểu đồ.

Tin tức cũng có thể tác động đến thị trường.

Ví dụ:

Bitcoin ETF news

Federal Reserve interest rates

Crypto regulation

Exchange hacked

New blockchain upgrade

Institutional adoption

Hệ thống cần có một module:

News Collector

có nhiệm vụ thu thập dữ liệu từ các nguồn phù hợp.

Sau đó chuẩn hóa thành:

```
News

id
title
content
source
publishedAt
crawledAt
relatedCoins
url
```

Ví dụ:

```
title:
Bitcoin rises after ...

publishedAt:
2026-07-28 08:15

relatedCoins:
BTC

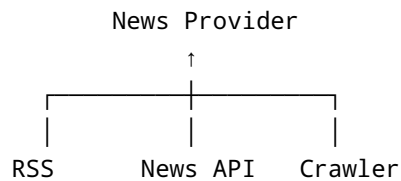
source:
XXX
```

28. News không được gắn cứng với một crawler

Không nên thiết kế:

Trading System
↓
Website A Crawler

Nên có:



Các provider trả về cùng một format chuẩn.

Ví dụ:

NewsItem

Nhờ đó việc thay nguồn dữ liệu không ảnh hưởng đến các module phía sau.

29. Module 11 – Sentiment Analysis

Sau khi có news:

"Bitcoin surges after institutional adoption..."

Machine Learning Service có thể phân loại:

POSITIVE

Tin:

"Major exchange suffers security breach..."

có thể là:

NEGATIVE

Tin trung lập:

"Bitcoin network upgrade scheduled..."

có thể là:

NEUTRAL

Kết quả lưu:

News

sentiment:
POSITIVE

score:
0.82

30. Sentiment có thể trở thành một Strategy

Đây là một điểm kiến trúc thú vị.

Ban đầu:

MA
RSI
BB
SR

Sau này có thể có:

NewsSentimentStrategy

Ví dụ:

Average sentiment trong 1 giờ > 0.7

→ BUY

Average sentiment < -0.7

→ SELL

Sau đó hệ thống có thể tìm:

MA

+

RSI

+

News Sentiment

hoặc:

Support Resistance

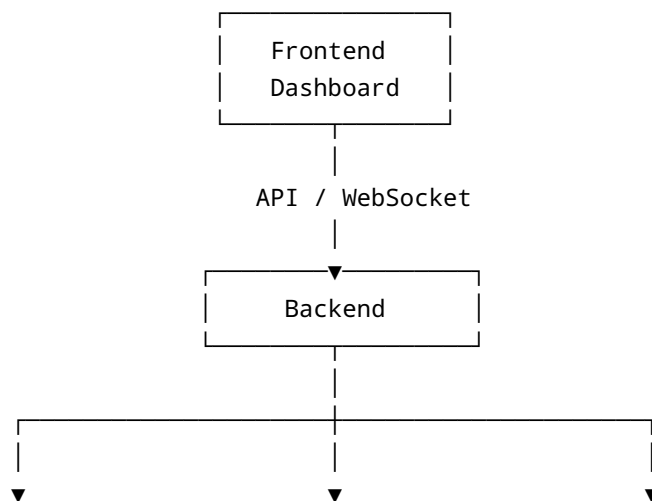
+

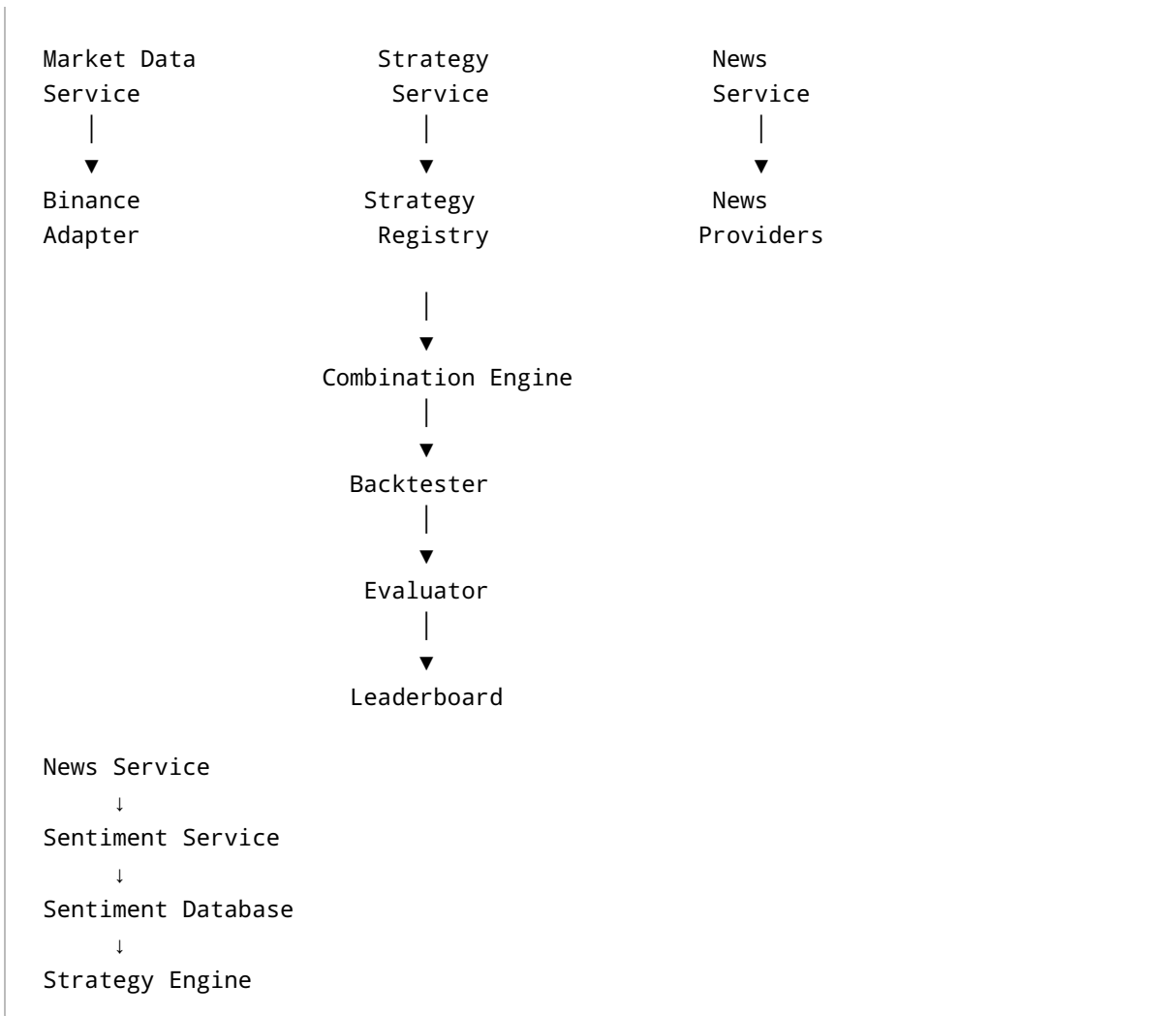
News Sentiment

Như vậy kiến trúc không còn giới hạn ở Technical Analysis.

31. Kiến trúc tổng thể gợi ý

Một kiến trúc logic có thể gồm:





Đây chỉ là kiến trúc tham khảo.

Nhóm được phép đề xuất kiến trúc khác nếu giải thích được lựa chọn của mình.

32. Những vấn đề Kiến trúc phần mềm mà đồ án phải giải quyết

Sinh viên cần xem đây là các **architectural drivers**.

32.1 Modifiability

Có thể thêm:

MACD Strategy

mà không phải sửa 20 module.

32.2 Scalability

Ban đầu:

10 strategies

Sau này:

100,000 candidate strategies

hệ thống có thể thay đổi kiến trúc như thế nào?

32.3 Realtime

Khi Binance có dữ liệu mới:

Market Data
→ Indicator
→ Strategy
→ UI

làm sao cập nhật với độ trễ thấp?

32.4 Reliability

Nếu Binance mất kết nối:

Connection lost

hệ thống xử lý ra sao?

Reconnect?

Retry?

Có mất candles không?

32.5 Performance

Có 1.000 strategy cần backtest.

Có nên chạy tuần tự:

```
1  
2  
3  
...  
1000
```

hay sử dụng:

```
Job Queue  
+  
Workers
```

32.6 Maintainability

Strategy Search không được phụ thuộc chặt vào Backtesting implementation.

Ví dụ có thể thay:

```
Random Search
```

bằng:

```
Genetic Search
```

mà Backtester vẫn giữ nguyên.

32.7 Observability

Hệ thống nên biết:

```
Loop đang chạy hay dừng?  
  
Đã thử bao nhiêu strategy?  
  
Backtest mất bao lâu?
```

Có bao nhiêu job lỗi?

Strategy nào đang đứng Top 1?

33. Một luồng hoàn chỉnh của hệ thống

Ví dụ:

Người dùng chọn:

BTCUSDT
5m

Bước 1 – Market Data

Binance
↓
BTC candles

Bước 2 – Strategy Generator

Sinh candidate:

MA20
+
RSI14
+
SupportResistance

Bước 3 – Backtester

Chạy trên:

BTCUSDT
01/01 → 01/07
5m

Bước 4 – Trade Simulation

Sinh:

82 trades

Bước 5 – Evaluator

Tính:

Return = 18.2%
Win Rate = 61%
MDD = -6.1%

Bước 6 – Ranking

Tính:

Score = 81.4

Bước 7

Candidate hiện tại đứng:

Rank #2

Bước 8

Frontend nhận event:

LEADERBOARD_UPDATED

Bước 9

Leaderboard tự cập nhật.

Không cần refresh trang.

34. Các Event có thể xuất hiện

Nhóm sử dụng event-driven architecture có thể định nghĩa:

```
MarketPriceUpdated  
  
CandleClosed  
  
StrategyGenerated  
  
BacktestStarted  
  
BacktestCompleted  
  
StrategyEvaluated  
  
LeaderboardUpdated  
  
NewsCollected  
  
SentimentAnalyzed
```

Ví dụ:

```
Backtest Worker
```

không cần gọi trực tiếp:

```
LeaderboardService.update()
```

mà có thể publish:

```
StrategyEvaluatedEvent
```

Ranking Service nhận event đó.

Điều này giúp giảm coupling giữa các module.

35. Database

Có thể có các nhóm dữ liệu:

Market Data

```
Candles  
Pair  
Timeframe  
Timestamp  
Open  
High  
Low  
Close  
Volume
```

Strategy

```
StrategyDefinition  
Parameters  
Version  
CreatedAt
```

Experiment

```
Combination  
Dataset  
Timeframe  
Parameters  
Result
```

Trades

```
Entry  
Exit  
Profit  
Strategy
```

News

Title
Content
Source
PublishedAt
RelatedCoin
Sentiment

Leaderboard

Có thể:

lưu trực tiếp

hoặc:

tính từ Experiment Results

Nhóm cần giải thích lựa chọn.

36. Strategy phải có Version

Ví dụ:

MA-RSI Strategy v1

MA20
MA50
RSI14

Sau đó sửa:

MA-RSI Strategy v2

MA10
MA30
RSI21

Không nên overwrite kết quả cũ.

Cần đảm bảo:

Experiment #122

luôn biết chính xác nó đã sử dụng strategy nào.

Đây là vấn đề:

Reproducibility.

37. Mức tối thiểu – MVP

Để tránh đồ án quá lớn, nhóm bắt buộc hoàn thành tối thiểu:

Market

- Binance data.
- Candlestick chart.
- Realtime update.
- Tối đa 4 timeframe.

Strategy

Ít nhất 4 strategy đơn lẻ, ví dụ:

MA
RSI
Bollinger
Support/Resistance

Combination

Có khả năng tạo composite strategy.

Backtest

Có khả năng giả lập giao dịch trên historical data.

Evaluation

Tối thiểu:

Return
Win Rate
Max Drawdown
Number of Trades

Search

Ít nhất một phương pháp:

Random Search

Leaderboard

Top-K strategies.

Visualization

Chart có:

Buy/Sell
Entry/Exit

News

Có pipeline:

Collect
↓
Store
↓
Analyze sentiment

38. Phần mở rộng

Các nhóm có thể mở rộng bằng:

Search

Genetic Algorithm
Evolutionary Search
Bayesian Optimization
LLM Strategy Generator

Trading

Long/Short
Stop Loss
Take Profit
Trailing Stop
Position Sizing

Market

Multiple Coins
Multiple Exchanges

ML

Sentiment
Price Prediction
Market Regime Detection

Architecture

Redis

Kafka/RabbitMQ

Worker Pool

Microservices

CQRS

Event Sourcing

Plugin Architecture

Không được cộng điểm chỉ vì sử dụng công nghệ phức tạp.

Nhóm phải chứng minh:

Công nghệ đó giải quyết vấn đề kiến trúc nào?

39. Một ví dụ để hiểu đúng mục tiêu đề án

Không nên hiểu đề án là:

Viết MA + RSI để kiếm tiền.

Mà phải hiểu là:

Thiết kế một hệ thống mà hôm nay có MA + RSI, ngày mai có thể thêm SMC, Wyckoff, Sentiment hoặc một strategy hoàn toàn mới mà kiến trúc cũ vẫn hoạt động.

Tương tự, hôm nay hệ thống dùng:

Random Search

ngày mai có thể thay bằng:

Genetic Algorithm

mà:

Backtester
Evaluator
Leaderboard
Visualization

không cần viết lại.

Đây mới là vấn đề của **Software Architecture**.

40. Câu hỏi kiến trúc trung tâm

Trong báo cáo, nhóm phải trả lời được các câu hỏi:

1. Strategy mới được thêm vào hệ thống như thế nào?

Ví dụ:

MACDStrategy

được thêm mà sửa những component nào?

2. Search algorithm mới được thêm như thế nào?

Từ:

Random Search

sang:

Genetic Search

có ảnh hưởng Backtesting Engine không?

3. Market Data Provider mới được thêm như thế nào?

Từ:

Binance

sang:

Binance + OKX

có phải sửa frontend không?

4. Nếu số backtest tăng từ 100 lên 100.000 thì kiến trúc thay đổi thế nào?

5. Nếu News Service bị lỗi thì Chart có còn chạy không?

6. Nếu Sentiment Model thay đổi thì Strategy Engine có bị ảnh hưởng không?

7. Nếu Binance WebSocket disconnect thì hệ thống phục hồi như thế nào?

8. Làm sao kiểm tra một kết quả trên Leaderboard được tạo ra bởi version strategy nào?

41. Scenario đánh giá khả năng mở rộng

Giảng viên có thể đặt yêu cầu:

Hệ thống hiện có MA, RSI, Bollinger và Support/Resistance. Hãy bổ sung MACD.

Nhóm tốt chỉ cần:

```
class MACDStrategy implements Strategy
```

và:

```
StrategyRegistry.register(MACDStrategy)
```

Nhóm thiết kế coupling cao có thể phải sửa:

```
Controller  
Backtester  
UI  
Database  
Combination Engine  
Evaluator
```

Đây là một minh chứng trực quan cho chất lượng kiến trúc.

42. Scenario đánh giá khả năng thay đổi

Một scenario khác:

Hiện tại:

```
RandomStrategyGenerator
```

Giảng viên yêu cầu thêm:

DomainGuidedStrategyGenerator

Ví dụ:

```
interface StrategyGenerator {  
    generate()  
}
```

có:

RandomGenerator
DomainGuidedGenerator
GeneticGenerator

Các component phía sau chỉ nhận:

CandidateStrategy

và không cần biết candidate được sinh ra bằng cách nào.

43. Scenario đánh giá scalability

Giả sử:

1 Backtest Worker

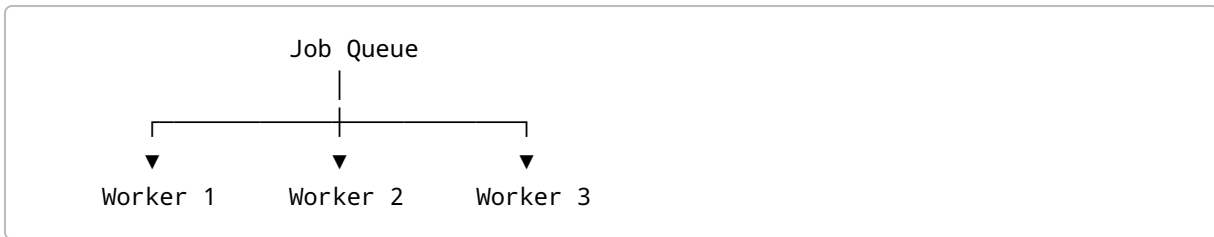
mất:

2 giây / candidate

10.000 candidate cần:

20.000 giây

Hệ thống nên cho phép mở rộng:



Đây là ví dụ để sinh viên hiểu tại sao:

Queue
Worker
Event

có thể quan trọng trong kiến trúc này.

44. Các Anti-pattern nên tránh

God Service

Không nên có:

TradingService

vừa:

```
get Binance data
calculate RSI
crawl news
run ML
backtest
rank
save database
send WebSocket
```

Hard-coded Strategy

Không nên:

```
if MA && RSI ...  
else if MA && Bollinger ...  
else if RSI && Bollinger ...
```

Frontend chứa business logic

Không nên để React/Vue tính:

```
trading strategy  
backtest  
profit  
ranking
```

Strategy truy cập trực tiếp Database

Không nên:

```
RSIStrategy  
↓  
MySQL
```

Strategy nên nhận data cần thiết thông qua abstraction thích hợp.

Crawler phụ thuộc chặt vào ML

Không nên:

```
Crawler → BERT model
```

Crawler chỉ cần:

```
collect news
```

Sentiment Service xử lý:

```
analyze news
```


45. Deliverables

Nhóm cần nộp:

1. Source Code

Repository hoàn chỉnh.

2. README

Hướng dẫn:

```
Install
Run
Architecture
Demo
```

3. Architecture Document

Tối thiểu mô tả:

```
System Context

Container/Module decomposition

Component responsibilities

Data Flow

Realtime Flow

Strategy Flow

Search/Backtest Flow
```

4. Architectural Decisions

Ví dụ:

```
ADR-001
Tại sao dùng WebSocket?

ADR-002
```

Tại sao dùng Plugin Architecture cho Strategy?

ADR-003

Tại sao dùng Queue cho Backtesting?

ADR-004

Tại sao tách Sentiment Service?

5. Demo

Demo tối thiểu:

Realtime chart

Multi timeframe

Thêm/chọn strategy

Generate combination

Backtest

Leaderboard

Trade visualization

News

Sentiment

46. Demo scenario đề xuất

Một demo tốt có thể diễn ra như sau.

Bước 1

Mở BTCUSDT.

5m | 15m | 1h | 4h

4 chart realtime.

Bước 2

Chọn:

MA
RSI
Bollinger
Support Resistance

Bước 3

Bấm:

START SEARCH

Bước 4

Màn hình hiển thị:

Candidates tested: 125

Current:
MA20 + RSI14 + SR

Backtesting...

Bước 5

Leaderboard thay đổi:

#1 MA20 + RSI14 + SR
#2 MA50 + BB
#3 RSI + SR

Bước 6

Click Top #1.

Chart hiển thị:

Buy
Sell
MA
Support
Resistance

Bước 7

Hiển thị:

Trades = 81
Win Rate = 61%
Return = 18.2%
MDD = -6.1%

Bước 8

Chuyển sang News:

BTC News

Positive: 42%
Neutral: 38%
Negative: 20%

Bước 9

Thêm:

SentimentStrategy

vào search space.

Bước 10

Chạy lại loop:

MA + RSI + Sentiment

MA + SR + Sentiment

...

Qua demo này có thể thấy hầu hết các component kiến trúc hoạt động cùng nhau.

47. Ý nghĩa cuối cùng của đề án

Đề án không nhằm chứng minh rằng:

MA + RSI + SMC

có thể kiếm tiền thật.

Mục tiêu là xây dựng một **software architecture** có khả năng thử nghiệm các ý tưởng như vậy một **cách có hệ thống**.

Hệ thống phải chuyển bài toán:

"Tôi có một strategy mới."

thành:

```
Plugin Strategy
  ↓
Combine
  ↓
Backtest
  ↓
Evaluate
  ↓
Compare
  ↓
Leaderboard
  ↓
Visualize
```

và có thể lặp lại quá trình:

Generate
→ Execute

- Measure
- Rank
- Improve
- Generate...

Do đó, bản chất đồ án là sự kết hợp của:

Realtime System + Plugin Architecture + Data Pipeline + Event-driven Architecture + Experiment Platform + Verification Loop.

Sinh viên được tự do lựa chọn framework, database, message queue, mô hình ML và thuật toán tìm kiếm.

Điều quan trọng nhất cần chứng minh là:

Kiến trúc được thiết kế như thế nào để các thành phần có thể thay đổi, mở rộng và hoạt động độc lập trong khi toàn bộ hệ thống vẫn duy trì được tính đúng đắn, khả năng quan sát và khả năng phát triển lâu dài.